LESSON 09 · MISSION GUIDE

The Living Ledger

pandas: the whole Roll of the Realm in one variable, with filters, sorting and Excel in and out.

From **Mistress Pell, Paymaster of the Guilds**
To **Jen, Keeper of the Ledger**
By **raven, regarding the fourteen tabs**

Keeper, you know the Treasury workbook. Fourteen tabs. "Roll", "Roll (2)", "Roll OLD DO NOT USE", "Pell working", "Dalvric working" (locked, password lost at sea). Every moon I filter, sort, add my helper columns, copy to a new tab, and pray to whichever saint looks after ledgers.

Orrin says pandas gives you a **DataFrame**: the whole Roll in one variable. Filter it, sort it, add columns, read it from Excel, write it back out, never click "Enable editing". A ledger that lives, he says. I said that sounds like something Dalvric would say. He went quiet. The real ledger is `data/payroll.xlsx`. Don't rename the sheets; the Grimoire still reads them. — P.

What you'll be able to do after today

- [ ] Read a CSV or an Excel sheet into a **DataFrame** and look at it: `.shape`, `.head()`, `.describe()`
- [ ] Pick a **column** (`df["salary"]`) or several (`df[["a", "b"]]`) and summarise it
- [ ] **Filter rows** the AutoFilter way, with masks, `.isin()` and `.str.contains()`
- [ ] **Add columns** computed from other columns, no dragging
- [ ] **Sort** by one or more columns, and find the top or bottom N
- [ ] Write a DataFrame to CSV, and turn a text column into real **dates**

Why this matters at work

- **This replaces the helper tabs.** Filter, sort, add columns: the things you do by hand every month are one line each, and the script does them identically next month.
- **Excel in, Excel out.** pandas reads the real workbook and writes files Excel opens. Nobody else has to change anything.

Four big ideas

1. DataFrame = sheet, Series = column	`df` is the table. `df["annual_salary"]` is one column, a Series, which is a NumPy array with a name and an index.
2. The index is the row label	The numbers down the left: start at 0, stick to their rows when you filter or sort. Not Excel row numbers.
3. Masks, exactly like NumPy	`df[df["annual_salary"] > 40000]`. Combine with `&` and `|`, brackets round each comparison.
4. Most things return a new DataFrame	`df.sort_values(...)` gives you a sorted copy; `df` itself is unchanged. Chain them: `df[mask].sort_values("x").head()`.

Three things that will bite you

- Column names are **case-sensitive** and must match exactly: `df["Location"]` is a KeyError if the column is `location`.
- `df["a", "b"]` is wrong; `df[["a", "b"]]` (a list inside the brackets) is right.
- `.to_csv(...)` without `index=False` writes the index as a column. You'll get "Unnamed: 0" back.

How today works

15 MIN
Lecture
Lars opens the Roll and does Mistress Pell's month in five lines

60 MIN
Exercise sheet
Seven ravens about the 200-row Roll, plus the Treasury workbook

15 MIN
Pages from the Grimoire, page 9
Dalvric's last ever script, note: "should work"

The ravens

#	FROM	RAVEN	YOU'LL USE
1	Pell	Open the file	`read_csv` `.shape` `.columns` `.head()` `.tail()`
2	Pell	One column at a time	Series, `.mean()` `.nunique()` `.describe()`
3	Siv	AutoFilter	masks, `.isin()` `.str.contains()`
4	Pell	New columns	`df["new"] = ...`
5	The Queen	Sorting	`.sort_values()`, `ascending=`
6	Orrin	Excel in, CSV out	`read_excel` `.to_csv`
7 ⭐	Pell	Service anniversaries	`pd.to_datetime` `.dt.month`

You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if the Queen's letters can wait), each printing the expected output
- [ ] Dalvric's headcount runs and prints the expected five lines, with one commit per bug
- [ ] Everything is pushed to `lesson-09` and you've opened a pull request



Next lesson: The Council of Guilds. `groupby` is the PivotTable, `merge` is XLOOKUP, and together they give you the pay gap by guild, by grade, by town. The Queen will want all three.